

High-Performance Adaptive Reverse Proxy & Load Balancer

Week-wise Task & Microtask Breakdown

Aditya Prakash | Roll No. 2302CS11 | Dual Degree (B.Tech + M.Tech), CSE | IIT Patna | Based on: BTP Proposal + Novelty/Adaptive Extensions

How this schedule is organized

This is a compressed 12-week schedule. Weeks 1-5 build the fixed-rule baseline system (proxy core, all 4 LB strategies, health checks, pooling, rate limiting, caching) — this alone matches the full scope of the original proposal and is a safe fallback deliverable on its own. Weeks 6-9 add observability, canary routing, and all four adaptive/novel components (Adaptive Load Balancing, Early Degradation Detection, Self-Tuning Canary Deployment, Adaptive Rate Limiting) — several are paired within the same week since they share the same underlying metrics pipeline. Weeks 10-12 run the fixed-vs-adaptive experiments and produce the final report, code, and demo. This timeline is tight: several weeks now carry two components instead of one, so treat the task lists as a hard sequence and start each week's tasks on day one rather than mid-week. Every week still ends in a concrete, demoable artifact so progress stays visible throughout.

PHASE 1 — Core Proxy & Load Balancing Foundation (Weeks 1-2)

Week 1 — Project Setup, TCP/HTTP Core & Single-Backend Forwarding

Stand up the environment and get a real, working reverse proxy forwarding to one backend by end of week.

- Finalize tech stack (Go or C++20), set up repo, Docker Compose with 2-3 dummy backends, and CI
- Implement an epoll/event-loop based non-blocking TCP listener
- Implement HTTP/1.1 request line, header, and body parsing (incl. chunked encoding)
- Implement forwarding to a single hardcoded backend and relay the exact response
- Add structured logging (request id, latency, status code) and basic integration tests

**WEEKLY
DELIVERABLE /
DEMO**

Proxy correctly forwards GET/POST requests to one backend and returns the exact response, demoed with curl and a browser.

Week 2 — Multi-Backend Pool + All 4 Load-Balancing Strategies

Generalize to a backend pool and implement every LB algorithm needed for later comparison.

- Design a backend-pool data structure and a pluggable LoadBalancer strategy interface
- Implement Round Robin and Weighted Round Robin
- Implement Least Connections (active-connection tracking) and IP Hash (sticky sessions)
- Add a config/flag to register N backends and switch strategy at runtime
- Write a distribution script that fires traffic and plots per-backend hit counts for all 4 strategies

**WEEKLY
DELIVERABLE /
DEMO**

Comparative demo: same traffic pattern run against all 4 strategies, showing distinct distribution graphs for each.

PHASE 2 — Reliability & Traffic Control (Weeks 3-5)

Week 3 — Active + Passive Health Checking & Failover

Make the proxy resilient to backend failure without manual intervention.

- Implement periodic active health-check probes (configurable interval/timeout)
- Implement passive failure detection from live traffic (consecutive errors/timeouts)
- Auto-remove unhealthy backends from LB rotation and auto-reinstate on recovery
- Write a failover-time measurement script (kill backend -> measure detection time)

WEEKLY DELIVERABLE / DEMO

Demo: kill one backend mid-traffic and show the proxy reroutes within one health-check interval; restart it and show automatic reinstatement.

Week 4 — Connection Pooling + Token-Bucket Rate Limiting

Add both a performance optimization (pooling) and a protection mechanism (rate limiting) in the same week since they touch the same request path.

- Implement a per-backend persistent connection pool with keep-alive reuse
- Benchmark latency with pooling on vs off
- Implement token-bucket rate limiting (per-client/global) with configurable refill rate and burst size
- Wire in 429 responses on exhaustion and add a burst-traffic test script

WEEKLY DELIVERABLE / DEMO

Two-part demo: (1) latency drop with connection pooling enabled, (2) burst script triggering 429s at the configured threshold.

Week 5 — In-Memory LRU Response Caching (Baseline Complete)

Cut backend load/latency for cacheable responses. This closes out the fixed-rule baseline system.

- Implement an LRU cache with configurable size, TTL, and cache-key derivation
- Wire cache lookups/writes into the request path with correct invalidation on expiry
- Add cache hit/miss metrics
- Freeze and tag this milestone as 'baseline v1' (matches original proposal's fixed-rule scope)

WEEKLY DELIVERABLE / DEMO

Demo: repeated identical requests show cache hits (near-zero backend latency) vs a cold cache miss; baseline system tagged in git.

PHASE 3 — Observability, Canary & Adaptive Intelligence (Weeks 6-9)

Week 6 — Load-Testing Tool + Metrics Dashboard

Build the benchmarking instrument and observability layer that every later week depends on.

- Build a Go-based concurrent load generator with constant, burst, and ramp-up patterns, reporting p50/p95/p99 latency, RPS, error rate
- Instrument the proxy with Prometheus-compatible metrics (latency, RPS, error rate, per-backend state)
- Stand up a live dashboard (Grafana or a lightweight custom one) driven by those metrics
- Validate both tools together against the Week 5 baseline proxy

WEEKLY DELIVERABLE / DEMO

Live dashboard demo showing real-time latency/throughput/per-backend health while the load tester runs, with a saved benchmark report.

Week 7 — Static Canary Routing + Adaptive Load Balancing

Add fixed-percentage canary routing as a control, then build the first and most central novel component.

- Implement weighted traffic splitting between 'stable' and 'canary' backend groups (e.g. 95/5) with separate metrics
- Design a scoring function combining latency, active connections, and error rate per backend
- Implement a feedback loop that periodically recomputes LB weights from live metrics, with min/max safety bounds
- Run a controlled experiment: slow one backend and show adaptive weights shifting traffic away from it

WEEKLY DELIVERABLE / DEMO

Side-by-side demo: fixed Round Robin vs Adaptive LB under an induced load skew, showing adaptive routing correcting automatically.

Week 8 — Early Degradation Detection + Self-Tuning Canary

Extend the adaptive signal pipeline from Week 7 into two more novel components that reuse it.

- Implement an EWMA/rolling-window tracker per backend to flag 'degrading' before hard failure
- On degradation, gradually reduce (not zero out) traffic share, with gradual restoration on recovery
- Define promote/rollback rules for the canary group based on live error rate and tail latency
- Implement automatic canary promotion (healthy) and automatic rollback (faulty), with event logging

WEEKLY DELIVERABLE / DEMO

Two demos: (1) synthetic backend slowdown triggers gradual traffic reduction before a hard failure would; (2) a deliberately broken canary auto-rolls-back while a healthy one auto-promotes.

Week 9 — Adaptive Rate Limiting + Full System Integration

Complete the last novel component and unify every fixed/adaptive module behind one control plane.

- Feed backend load/health signals into the rate limiter so limits tighten under stress and relax on recovery
- Add hysteresis/cooldown to prevent oscillation
- Integrate adaptive LB, degradation detection, self-tuning canary, and adaptive rate limiting into one control loop
- Add a global 'fixed mode' vs 'adaptive mode' runtime toggle for clean A/B comparison
- Freeze and tag milestone as 'adaptive v1' (feature-complete)

WEEKLY DELIVERABLE / DEMO

End-to-end demo of the complete system with a single switch to flip between fixed and adaptive behaviour live, on the dashboard.

PHASE 4 — Evaluation, Documentation & Delivery (Weeks 10-12)

Week 10 — Comprehensive Benchmarking Runs

Generate the experimental data the entire comparison story depends on.

- Design experiment matrix: {RR, WRR, Least-Conn, IP Hash, Adaptive} x {constant, burst, ramp-up} traffic
- Run fixed-vs-adaptive canary experiments (healthy and faulty canary scenarios)
- Run fixed-vs-adaptive rate-limiting experiments under induced backend stress
- Capture p50/p95/p99 latency, throughput, error rate, failover time, and CPU/memory for every run, with metadata for reproducibility

WEEKLY DELIVERABLE / DEMO

Complete raw benchmark dataset covering every strategy and traffic pattern, checked into the repo.

Week 11 — Analysis, Visualization & Documentation

Turn raw numbers into report-ready results and make the codebase presentable — combined into one week given the tighter schedule.

- Generate throughput-vs-concurrency and latency-percentile (fixed vs adaptive) comparison plots
- Build failover-time and resource-usage comparison tables, and draft the 'Results' narrative
- Clean up code, add module READMEs and architecture diagrams
- Finalize Docker Compose/Kubernetes manifests for one-command demo setup, and tag the final release

WEEKLY DELIVERABLE / DEMO

Finalized comparison charts/tables ready for the report, plus a documented repo a stranger could clone and run end-to-end.

Week 12 — Final Report, Slides & Viva Prep

Package everything into the final academic deliverables and rehearse the defense.

- Write the final BTP report (objectives, architecture, adaptive design, results, conclusion)
- Prepare presentation slides with architecture diagrams and key result plots
- Prepare a live/recorded demo script covering fixed-vs-adaptive behaviour
- Do a mock viva/Q&A; run-through and use remaining time as a fix-it buffer

WEEKLY DELIVERABLE / DEMO

Submission-ready final report, slide deck, and rehearsed live demo.

Summary: Milestone Checkpoints

Checkpoint	Week	What it proves
Baseline proxy feature-complete	5	Matches full scope of original proposal: 4 LB strategies, health checks, pooling, rate limiting, caching
Observability + canary ready	6	Load tester + live dashboard operational
Adaptive layer feature-complete	9	All 4 novel adaptive components integrated with a fixed/adaptive mode switch
Experimental results finalized	11	Full fixed-vs-adaptive comparison data and plots ready
Final delivery	12	Report, slides, demo, and reproducible open-source repo submitted

Metrics tracked throughout (Week 6 onward): p50/p95/p99 latency, requests/sec, error rate, failover time, and CPU/memory usage — captured consistently so every weekly demo also contributes data toward the final fixed-vs-adaptive comparison in Weeks 10-11.